

TEHNIČKA DOKUMENTACIJA ARHITEKTURE I KODA SISTEMA FIT ORCHESTRATOR

Autor: Harun Riđević

Kategorija: AI :: FIT Agents

Dokumentacija Koda

1. Arhitektura sistema i struktura projekta

Softverski sistem FIT Orchestrator projektovan je i realizovan primjenom naprednih principa modularne, distribuirane i asinkrone arhitekture unutar programskog jezika Python. Arhitektonski skelet cjelokupnog rješenja oslanja se na specijalizovani radni okvir LangGraph, koji omogućava modelovanje inteligentnih sistema u obliku stanja i operacija unutar usmjerenog acikličnog grafa (DAG). Za razliku od konvencionalnih i rigidnih linearnih LLM lanaca koji podatke procesiraju isključivo jednosmjerno i sekvencijalno, LangGraph uvodi mogućnost uvođenja kontrolisanih povratnih petlji (iteracija) i dinamičkog uslovnog grananja. Ova karakteristika je od esencijalnog značaja za kompleksne kognitivne zadatke evaluacije, samokorekcije i automatskog ispravljanja grešaka u realnom vremenu, čime se emulira ljudski način rješavanja problema.

Projektna struktura izvornog koda je radi lakšeg održavanja i skalabilnosti strogo podijeljena u dva primarna operativna modula:

- **Modul `agents.py` (Backend):** Predstavlja logičko jezgro aplikacije. Ovaj fajl inkapsulira cjelokupnu poslovnu logiku, deklaraciju topologije grafa, matematičke i logičke metode memorijskog sloja, te direktnu integraciju sa jezičkim modelima i eksternim pretraživačkim API servisima.
- **Modul `app.py` (Frontend):** Implementiran je korištenjem Streamlit radnog okvira. Njegova primarna uloga je renderovanje reaktivnog korisničkog interfejsa, vizuelna reprezentacija tekućih agencijskih procesa, dinamičko praćenje stanja mreže i strukturirani prikaz generisanih izvještaja.

2. Implementacija dijeljenog stanja (ResearchState)

Srž komunikacije, očuvanja konzistentnosti podataka i eliminacije neželjenih nuspojava među izolovanim agentima predstavlja klasa `ResearchState`, koja nasljeđuje bazičnu Python `TypedDict` strukturu. Unutar ovog objekta skladišti se cjelokupan globalni operativni kontekst sistema, što znači da se stanje ponaša kao centralna memorijska magistrala aplikacije. Atributi unutar klase su striktno tipizirani i obuhvataju:

- `topic`: Tekstualni string koji pohranjuje primarni korisnički upit.
- `search_queries`: Dinamička lista stringova sa optimizovanim upitima za pretragu.
- `search_results`: Lista strukturiranih rječnika u koje se smještaju URL adrese i sirovi tekstualni isječci prikupljeni sa weba.
- `academic_analysis`, `fact_check`, `final_report`: Tekstualni stringovi velikog kapaciteta namijenjeni za pohranu rezultata međufaza i konačnog dokumenta.

Pored primarnih podataka, `ResearchState` inkapsulira i ključne metapodatke sistema. Posebno je značajan atribut `messages` koji koristi operator pridruživanja `operator.add`. Ovaj mehanizam osigurava da se svaka nova agencijska poruka atomski nadodaje na postojeću listu, čime se formira nepromjenjivi (immutable) hronološki log aktivnosti. Atributi `current_step`, `reflection_score`, `reflection_feedback` i `reflection_attempts` služe za telemetriju i preciznu orkestraciju kontrolnih petlji. Ovakav dizajn garantuje da svaki registrovani čvor u grafu ima trenutani, thread-safe pristup cjelokupnoj historiji izvršavanja bez potrebe za eksternim bazama ili kompleksnim sistemima za razmjenu poruka.

3. Mehanizam stabilnosti i API integracija

Direktna komunikacija sa eksternim infrastrukturnim servisima, specifično sa Groq API-jem i jezičkim modelom llama-3.1-8b-instant, osigurana je kroz prilagođenu zaštitnu funkciju (wrapper) pod nazivom `safe_llm_invoke()`. U produkcijskim uslovima, jedan od najvećih problema pri radu sa LLM modelima visokih performansi jesu striktna ograničenja frekvencije upita u minuti i sekundi (Rate Limits). Kako bi se ovaj problem neutralisao, funkcija `safe_llm_invoke()` presreće mrežne izuzetke i greške tipa HTTP 429 te automatski pokreće napredni algoritam eksponencijalnog odgađanja (Exponential Backoff) sa slanjem ponovnih upita (retries).

Ukoliko se detektuje preopterećenje API-ja, nit izvršavanja se privremeno uspavljuje, a vrijeme čekanja se dinamički izračunava prema matematičkoj formuli $Wait = Delay \times 2^{\{attempt\}}$. Unutar maksimalno 4 definisana pokušaja, sistem sekvencijalno pauzira rad na 5, 10, 20 i konačno 40 sekundi prije nego što proglasi kritičnu grešku. Ovaj inženjerski pristup drastično podiže nivo otpornosti sistema na mrežne anomalije (Fault Tolerance), efikasno apsorbuje vršna opterećenja i programski garantuje uspješan i deterministički završetak svakog započetog ciklusa unutar grafa.

4. Hibridni memorijski sistem i semantička pretraga

FIT Orchestrator posjeduje dugoročnu, perzistentnu memoriju realizovanu kroz hibridni dvoslojni mehanizam pohrane koji simulira ljudsku kognitivnu akumulaciju iskustva tokom vremena. Vektorski sloj memorije oslanja se na lokalno instanciranu bazu podataka ChromaDB i embedding model `all-MiniLM-L6-v2`. Prilikom pokretanja nove sesije, funkcija `find_related_memories_vector()` pretvara tekstualni unos korisničke teme u visokodimenzionalni numerički vektor i vrši matematički proračun kosinusne sličnosti (Cosine Similarity) u odnosu na sve istorijski sačuvane sesije. Ukoliko izračunata vrijednost prelazi prag relevantnosti od **0.25**, podaci se de-serijalizuju i ubrizgavaju u radni kontekst kao dodatno znanje, što dramatično redukuje tendenciju LLM modela ka halucinacijama. Ako je baza nedostupna uslijed hardverskih ograničenja, sistem aktivira robusni *fallback* podsistem koji vrši tokenizaciju i preklapanje ključnih riječi (Keyword Overlap) na bazi lokalnog JSON fajla.

Upis novih spoznaja izvršava se asinkrono unutar funkcije `update_memory_after_research()` nakon što je konačni izvještaj uspješno generisan. Kako bi se spriječilo dupliciranje identičnih unosa, sistem kreira unikatni ključ (ID) primjenom MD5 hash algoritma nad konkatenacijom naziva teme i vremenske oznake, a zatim vrši *upsert* naredbu u ChromaDB kolekciju. Paralelno s tim, ažurira se i strukturirana lokalna datoteka `agent_memory.json` koja bilježi statističke obrasce upita (query patterns) mapirane po specifičnim semantičkim domenama i vodi hronološki dnevnik učenja (`learning_log`), koji se automatski skraćuje na 50 najnovijih unosa radi optimizacije diskovnog prostora.

5. Multi-Agent Mreža i Cjevovod (Pipeline)

Proces dekonstrukcije, pretrage, analize i sinteze podataka unutar sistema FIT Orchestrator distribuiran je na 6 visoko specijalizovanih agencijskih čvorova koji funkcionišu kao autonomne komponente unutar cjevovoda:

- **Orchestrator Agent:** Inicijalni kognitivni čvor grafa. Analizira primarnu temu i istorijski memorijski kontekst, kreira verbalnu strategiju istraživanja i generiše 4 unikatna upita za pretragu. Izlazni podaci se filtriraju regularnim izrazima (RegEx) kako bi se osiguralo pravilno JSON parsiranje.
- **Web Search Agent:** Komunicira sa naprednim interfejsom TavilySearchResults. Agent asinkrono izvršava generisane upite, preuzima sirovi tekst sa identifikovanih web lokacija i limitira dužinu sadržaja na 800 karaktera po izvoru, štiteći kontekstualni prozor jezičkog modela od prepunjavanja (Token Overflow).
- **Academic Agent:** Predstavlja analitičku tačku grafa. Pokreće se sa sistemski sniženom temperaturom (0.2) radi postizanja maksimalne objektivnosti. Sintetizuje prikupljene eksterne podatke sa istorijskim znanjem iz fusnota i memorije, te kreira rigoroznu akademsku analizu.

- **Self-Reflection Agent:** Automatizovani i nezavisni kritičar kvaliteta analize. Vrednuje rad akademskog agenta na osnovu četiri fiksna kriterija: dubina pokrivenosti teme, kvalitet argumenata, strukturalna jasnoća i inovativnost, te vraća JSON objekat sa numeričkom ocjenom (0-10) i kritikama.
- **Fact Check Agent:** Specijalista za verifikaciju i evaluaciju tačnosti. Izoluje ključne tvrdnje iz analitičkog teksta, upoređuje ih sa URL adresama iz pretrage, dodjeljuje im nivoe pouzdanosti i računa ukupni procenat tačnosti dokumenta.
- **Writer Agent:** Završni kompajlerski čvor koji vrši stilsku sintezu analitičkih tekstova i podataka o provjeri činjenica u sveobuhvatan finalni izvještaj formatiran u Markdown sintaksi, te pokreće proces ažuriranja dugoročne memorije.

6. Kontrola toka i LangGraph uslovno rutiranje

Topologija i dinamika kretanja informacija kroz multi-agent sistem definisani su unutar metode `build_graph()`. Nakon što se svi agencijski čvorovi registruju pomoću funkcije `graph.add_node()`, uspostavljaju se operativne veze. Najveća arhitektonska prednost ovog sistema u odnosu na standardne AI sekvence ogleda se u implementaciji uslovnog rutiranja (Conditional Routing) pomoću funkcije `should_retry_analysis(state)`. Ova funkcija predstavlja uslovnu ivicu (Conditional Edge) grafa pozicioniranu neposredno nakon čvora za samorefleksiju.

Ukoliko funkcija evaluacijom stanja utvrdi da je parametar `reflection_score` strogo manji od kritične vrijednosti **7**, a da broj prethodnih pokušaja (`reflection_attempts`) još uvijek nije dostigao limit od **2**, graf donosi odluku o dinamičkom vraćanju toka izvršavanja unazad na čvor `academic` (`retry` grana). Prilikom povratka, tekstualne kritike iz polja `reflection_feedback` ubrizgavaju se kao obavezujući ulazni parametri za akademskog agenta, primoravajući ga da u novoj iteraciji korigira sve uočene nedostatke. U suprotnom, kada je uslov zadovoljen, tok se usmjerava naprijed ka čvorovima za provjeru činjenica i pisanje, čime se programski garantuje visok kvalitet izlaznih podataka.

7. Reaktivni frontend interfejs (Streamlit)

Korisnički interfejs sistema, implementiran unutar programskog modula `app.py`, razvijen je korištenjem Streamlit radnog okvira, uz napredne modifikacije kroz direktno ubrizgavanje prilagođenog HTML i CSS koda preko parametra `unsafe_allow_html=True`. Grafički interfejs definiše stilizovanu tamnu temu (Cyberpunk Dark Aesthetic) zasnovanu na dubokim tonovima sive, ljubičaste i cijan boje, uz upotrebu modernih Google fontova *Syne* i *DM Sans*, čime se postiže visok nivo vizuelne profesionalnosti.

Aplikacija je u potpunosti programirana da prati reaktivni obrazac ponašanja. Dok pozadinski graf asinkrono procesira podatke i izvršava agencijske zadatke, korisnički interfejs u realnom vremenu presreće promjene unutar stanja i dinamički ažurira CSS klase na ekranu. Korisnik vizuelno prati napredak kroz animirane indikatore skeniranja kartica agenata, pulsiranje aktivnih čvorova i reaktivnu traku progresu. Po završetku cjelokupnog ciklusa, podaci se organizuju kroz tabove (`st.tabs`), pružajući jasan i strukturiran uvid u konačni izvještaj, sirovu analizu, metriku samorefleksije sa brojem ostvarenih iteracija, provjeru činjenica, kao i uvid u stanje perzistentne memorije. Sve vizuelne komponente zajedno čine proces donošenja odluka unutar vještačke inteligencije transparentnim i razumljivim za krajnjeg posmatrača.